

```
import numpy as np
import matplotlib.pyplot as plt
import json
import sys
```

Introduction to numerical methods

Lecture: János Török Practice: Bendegúz Nyári, Kristóf Benedek

Class 1: Numbers

Required knowledge

I assume you all know the following concepts in python:

loops and conditions

```
#range runs from 0 to n-1
for i in range(4):
    print(i)
```

```
0
1
2
3
```

```
#from, to, steps
for a in range(3,9,2):
    print(a)
```

```
3
5
7
```

```
#indentation indicate code blocks (here loop and condition)
for a in range(9):
    if a < 5:
        print(a)
    else:
        print(-a)
```

```
0
1
2
```

```
3
4
-5
-6
-7
-8
```

#same in two ways:

```
for i in range(9):
    if i > 5:
        print(i)
print("#####")
for i in range(9):
    if i <=5: continue
    print(i)
```

*#The first one is preferred by purists, but in python the second is also good practice if used properly,
to avoid excess indentations*

```
6
7
8
#####
6
7
8
```

```
#while loop
i = 0
while i < 5:
    i += 1
    print(i)
print("END",i)
```

```
1
2
3
4
5
END 5
```

#while loop exit on condition

```
i = 0
while True:
    i += 1
    if i > 4:
        break
print("END",i)
```

```
END 5
```

Variable types

Integers, floating point, string, list, set, dictionary

integer

```
# Integer
a = 4
print(type(a))
print(a//2, a//3, a//4, a//5)
#modulus
print("modulus:",a%3,a%7)
# power
print("power:",2**3)

<class 'int'>
2 1 1 0
modulus: 1 4
power: 8
```

floating point

```
# Floating point
b = 2.2
c = 1.0
d = 1e-10
print(b,c,d)
print(type(c))

2.2 1.0 1e-10
<class 'float'>

# Back and forth to integer
print(a/1,float(a),int(b))

4.0 4.0 2
```

string

```
#string
s = "alma"
t = "korte"
print(s,t,s+t,2*s)
print(type(s))

alma korte almakorte almaalma
<class 'str'>

#not everything is possible:
print(t+1)
```

```

-----
-----
TypeError                                Traceback (most recent call
last)
<ipython-input-12-0463afbcebe8> in <module>
      1 #not everything is possible:
----> 2 print(t+1)

TypeError: can only concatenate str (not "int") to str

```

list

```

#lists contain arbitrary elements one after the other
l = [a,b,t]
print(l)
print(type(l))

[4, 2.2, 'korte']
<class 'list'>

#lists may contain lists as elements
l2 = [c,l]
print(l2)

[1.0, [4, 2.2, 'korte']]

#referencing them
print(l2[0])
print(l2[1])
print(l2[1][1])

1.0
[4, 2.2, 'korte']
2.2

#list comprehension
l = [i**2 for i in range(2,6)]
print(l)
l = [i**2 for i in range(2,6) if i%2 == 0]
print(l)

[4, 9, 16, 25]
[4, 16]

#making list from a range
print(range(5))
print(list(range(5)))

range(0, 5)
[0, 1, 2, 3, 4]

```

#adding and removing elements from the list

```
l = list(range(5))
l.append("8")
print("Append:",l)
l.remove(2)
print("Remove:",l)
```

Append: [0, 1, 2, 3, 4, '8']

Remove: [0, 1, 3, 4, '8']

#loop through elements

```
for le in l:
    print(le)
```

0
1
3
4
8

#loop with index

```
for i,le in enumerate(l):
    print(i,le)
for i in range(len(l)):
    print(i,l[i])
```

0 0
1 1
2 3
3 4
4 8
0 0
1 1
2 3
3 4
4 8

```
l.remove(2)
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
<ipython-input-21-2110d8e9703c> in <module>
```

```
----> 1 l.remove(2)
```

ValueError: list.remove(x): x not in list

```
x = 3
if x in l:
    l.remove(x)
```

```
else:
    print("the list does not contain x")
print(l)
```

```
[0, 1, 4, '8']
```

#removes the last element and returns it

```
m = l.pop()
print(l,m)
```

```
[0, 1, 4] 8
```

#remove element i

```
l = list(range(5))
m = l.pop(2)
print(m,l)
```

```
2 [0, 1, 3, 4]
```

#remove element i

```
l = [5,2,8,2,9,"4"]
i = l.index(2)
print(m,i)
```

```
2 1
```

#index gives an error if the item is not in the list so again we have to check with 'in'

```
print(l.index(4))
```


```
ValueError                                Traceback (most recent call last)
```

```
<ipython-input-26-f0baacf7323c> in <module>
```

```
    1 #index gives an error if the item is not in the list so again we have to check with 'in'
```

```
----> 2 print(l.index(4))
```

```
ValueError: 4 is not in list
```

#referencing ranges:

```
l = list(range(10))
print(l)
print(l[3:4]) # =[3] first included, the last isn't
print(l[-4:-3]) # =[6] - goes backwards, -1 is 9 here -2 is 8 -3 is 7, and -4 is 6 which is the result
print(l[4:3], l[-3:-4]) # [],[] both are empty, as the first index is larger than the second
print(l[1:7:2]) # =[1, 3, 5] from 1 to 7 (not included), by steps of 2
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[3]
[6]
[] []
[1, 3, 5]
```

tuple similar to list but cannot be changed and can be indexed, see dictionary

```
print(type([1,2]),type((1,2)))
```

```
<class 'list'> <class 'tuple'>
```

```
lt = (1,2)
```

```
print(lt,lt[1])
```

```
(1, 2) 2
```

```
lt[1] = 1
```

```
-----
```

```
TypeError                                Traceback (most recent call
last)
```

```
<ipython-input-30-390da2163dde> in <module>
```

```
----> 1 lt[1] = 1
```

```
TypeError: 'tuple' object does not support item assignment
```

```
#sort lists
```

```
print(sorted([8,3,7,4,9]))
```

```
print(sorted([8,3,7,4,9],reverse=True))
```

```
[3, 4, 7, 8, 9]
```

```
[9, 8, 7, 4, 3]
```

copying

```
l = [1,2,3]
```

```
k = l
```

```
k[0] = 7
```

```
print(k,l) # they are the same
```

```
[7, 2, 3] [7, 2, 3]
```

```
l = [1,2,3]
```

```
k = l.copy()
```

```
k[0] = 7
```

```
print(k,l) # they are not the same
```

```
[7, 2, 3] [1, 2, 3]
```

set

#similar to list but all element may appear only once

```
s = set()
l = list()
l.append(1)
l.append(1)
l.append(2)
s.add(1)
s.add(1)
s.add(2)
print(l,s)
print(set(l))
print(type(s))
```

```
[1, 1, 2] {1, 2}
{1, 2}
<class 'set'>
```

#standard set operations

#union

```
s1 = set([1,2,3])
s2 = set([3,4,5])
print(s1.union(s2))
print(s1.intersection(s2))
#etc...
```

```
{1, 2, 3, 4, 5}
{3}
```

dictionary List referenced by arbitrary hashable type, e.g. integer string, tuple, etc.

```
d = dict()
d[15643] = "2^14"
d["2^14"] = 16384
d["more"] = dict()
d["more"] = "Second level"
print(d)
```

```
{15643: '2^14', '2^14': 16384, 'more': 'Second level'}
```

#dictionary is just json, print it out nicely

```
print(json.dumps(d, indent=4))
```

```
{
  "15643": "2^14",
  "2^14": 16384,
  "more": "Second level"
}
```



```
#from string to dictionary:
s = '{"a":1,"b":2}'
d = json.loads(s)
print(d,type(d))

{'a': 1, 'b': 2} <class 'dict'>
```

```
#tuple as index
d[(1,2)] = [1,2]
print(d)

{'a': 1, 'b': 2, (1, 2): [1, 2]}
```

```
#the last one does not work other way around
d[[1,2]] = (1,2)
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
<ipython-input-38-6d429214a353> in <module>
      1 #the last one does not work other way around
----> 2 d[[1,2]] = (1,2)
```

```
TypeError: unhashable type: 'list'
```

```
#tuple is not in json
print(d)
print("#####")
print(json.dumps(d, indent=4))

{'a': 1, 'b': 2, (1, 2): [1, 2]}
#####
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
<ipython-input-39-af2b76b762b9> in <module>
      2 print(d)
      3 print("#####")
----> 4 print(json.dumps(d, indent=4))
```

```
/usr/lib/python3.8/json/__init__.py in dumps(obj, skipkeys,
ensure_ascii, check_circular, allow_nan, cls, indent, separators,
default, sort_keys, **kw)
    232     if cls is None:
    233         cls = JSONEncoder
--> 234     return cls(
    235         skipkeys=skipkeys, ensure_ascii=ensure_ascii,
    236         check_circular=check_circular, allow_nan=allow_nan,
```

```

indent=indent,

/usr/lib/python3.8/json/encoder.py in encode(self, o)
    199         chunks = self.iterencode(o, _one_shot=True)
    200         if not isinstance(chunks, (list, tuple)):
--> 201             chunks = list(chunks)
    202         return ''.join(chunks)
    203

/usr/lib/python3.8/json/encoder.py in _iterencode(o,
_current_indent_level)
    429         yield from _iterencode_list(o,
_current_indent_level)
    430     elif isinstance(o, dict):
--> 431         yield from _iterencode_dict(o,
_current_indent_level)
    432     else:
    433         if markers is not None:

/usr/lib/python3.8/json/encoder.py in _iterencode_dict(dct,
_current_indent_level)
    374         continue
    375     else:
--> 376         raise TypeError(f'keys must be str, int,
float, bool or None, '
    377                               f'not
{key.__class__.__name__}')
    378         if first:

```

TypeError: keys must be str, int, float, bool or None, not tuple

```

#delete it and check again:
del(d[(1,2)])
print(json.dumps(d, indent=4))

```

```

{
    "a": 1,
    "b": 2
}

```

```

dd = dict()
dd[np.int(2)] = np.complex128(1+2.3j)
print(json.dumps(dd, indent=4))

```

```

-----
-----
TypeError                                Traceback (most recent call
last)

```

```

<ipython-input-43-d24867ae86ef> in <module>
      1 dd = dict()
      2 dd[np.int(2)] = np.complex128(1+2.3j)

```

```

----> 3 print(json.dumps(dd, indent=4))

/usr/lib/python3.8/json/__init__.py in dumps(obj, skipkeys,
ensure_ascii, check_circular, allow_nan, cls, indent, separators,
default, sort_keys, **kw)
    232     if cls is None:
    233         cls = JSONEncoder
--> 234     return cls(
    235         skipkeys=skipkeys, ensure_ascii=ensure_ascii,
    236         check_circular=check_circular, allow_nan=allow_nan,
indent=indent,

/usr/lib/python3.8/json/encoder.py in encode(self, o)
    199         chunks = self.iterencode(o, _one_shot=True)
    200         if not isinstance(chunks, (list, tuple)):
--> 201             chunks = list(chunks)
    202         return ''.join(chunks)
    203

/usr/lib/python3.8/json/encoder.py in _iterencode(o,
_current_indent_level)
    429         yield from _iterencode_list(o,
_current_indent_level)
    430     elif isinstance(o, dict):
--> 431         yield from _iterencode_dict(o,
_current_indent_level)
    432     else:
    433         if markers is not None:

/usr/lib/python3.8/json/encoder.py in _iterencode_dict(dct,
_current_indent_level)
    403         else:
    404             chunks = _iterencode(value,
_current_indent_level)
--> 405             yield from chunks
    406             if newline_indent is not None:
    407                 _current_indent_level -= 1

/usr/lib/python3.8/json/encoder.py in _iterencode(o,
_current_indent_level)
    436             raise ValueError("Circular reference
detected")
    437             markers[markerid] = o
--> 438             o = _default(o)
    439             yield from _iterencode(o, _current_indent_level)
    440             if markers is not None:

/usr/lib/python3.8/json/encoder.py in default(self, o)
    177
    178     """

```

```

--> 179         raise TypeError(f'Object of type
{o.__class__.__name__} '
180                        f'is not JSON serializable')
181

TypeError: Object of type complex128 is not JSON serializable

#get parts:
print(d.items())
print(d.keys())
print(d.values())

dict_items([('a', 1), ('b', 2)])
dict_keys(['a', 'b'])
dict_values([1, 2])

for k,v in d.items():
    print(k,":",v)

a : 1
b : 2

d = dict()
d["a"] = 27
d["g"] = 18
d["bcbc"] = 355
d["af"] = 99

#create a tuple which is sorted by value
sorted_by_value = sorted(d.items(), key=lambda kv: kv[1])
print(sorted_by_value)

[('g', 18), ('a', 27), ('af', 99), ('bcbc', 355)]

```

Numbers

How numbers are represented and what is a numerical error

```

# numbers are stored in binary format
for i in range(10):
    print(i,bin(i))

0 0b0
1 0b1
2 0b10
3 0b11
4 0b100
5 0b101
6 0b110

```

```

7 0b111
8 0b1000
9 0b1001

# negative numbers
for i in range(10):
    print(-i, bin(-i))

0 0b0
-1 -0b1
-2 -0b10
-3 -0b11
-4 -0b100
-5 -0b101
-6 -0b110
-7 -0b111
-8 -0b1000
-9 -0b1001

#author: DeanM@stackoverflow
def int2bin(integer, digits):
    if integer >= 0:
        return bin(integer)[2:].zfill(digits)
    else:
        return bin(2**digits + integer)[2:]

# 8 bit integer
for i in range(10):
    print(i, int2bin(i, 8))

0 00000000
1 00000001
2 00000010
3 00000011
4 00000100
5 00000101
6 00000110
7 00000111
8 00001000
9 00001001

s = "00000110" # the problem is s is indexed from the left not from
the right
a = 0
j = 1
for i in reversed(range(len(s))): # reversed goes backwards
    if s[i] == "1":
        a += j
        j *= 2
print("a:", a % (2**8-1)) # we work with 8 bit numbers so get rid of
the rest

```

```

a = a % (2**8-1)
if a >= 2**7-1:
    print(a-2**8)
else:
    print(a)

a: 6
6

# 8 bit integer
for i in range(10):
    print(-i,int2bin(-i,8))

0 00000000
-1 11111111
-2 11111110
-3 11111101
-4 11111100
-5 11111011
-6 11111010
-7 11111001
-8 11111000
-9 11110111

s = "11111010" # the problem is s is indexed from the left not from
the right
a = 0
j = 1
for i in reversed(range(len(s))): # reversed goes backwards
    if s[i] == "1":
        a += j
        j *= 2
print("a:",a & (2**8-1)) # a better and more traditional way of going
back to 8 bit works only with numbers 2**n
a = a & (2**8-1)
if a >= 2**7-1:
    print(a-2**8)
else:
    print(a)

a: 250
-6

sys.int_info
#only 30 bits of 32 are used to store the integer the other two are
type information

sys.int_info(bits_per_digit=30, sizeof_digit=4)

```

Floating point

Representation There are three parts: sign s , exponent e and fraction f . The value:

$$n = (-1)^s \cdot (1+f) \cdot 2^{e-1023}$$

The sign is $s=1$, the exponent in decimal is $e=1 \cdot 2^{10} + 1 \cdot 2^1 = 1024 + 2 = 1026$, the fraction is $f = 1 \cdot \frac{1}{2^1} + 0 \cdot \frac{1}{2^2} \dots = 0.5$. Therefore $n = (-1)^1 \cdot (1+0.5) \cdot 2^{1026-1023} = -1 \cdot 1.5 \cdot 2^3 = -12.0$ (base10).

Problems

From 64 (actually 62) bit floating point numbers there are maximum $2^{62} \approx 4.6 \cdot 10^{18}$ different numbers. So if you write 19 digits it cannot be stored accurately. However the situation is much worse.

```
if 0.1+0.2==0.3:
    print("Math is ok")
else:
    print("This should not happen!")
```

This should not happen!

`0.1+0.2-0.3`

`5.551115123125783e-17`

```
a = 0.1
b = 0.2
c = 0.3
if abs(a+b-c) < 1e-16:
    print("Math is ok with some small error")
else:
    print("This should not happen!")
```

Math is ok with some small error

```
a = 0.1
b = 0.2
c = 0.3
s = 0.0
for _ in range(1000):
    s += a
    s += b
for _ in range(1000):
    s -= c
if abs(s) < 1e-16:
    print("Math is ok with some small error", s)
else:
    print("This should not happen!", s)
```

This should not happen! `-1.139943694994372e-11`

```

a = 0.1
b = 0.2
c = 0.3
s = 0.0
for _ in range(1000):
    s += a
    s += b
    s -= c
if abs(s) < 1e-16:
    print("Math is ok with some small error", s)
else:
    print("This should not happen!", s)

```

Math is ok with some small error 5.551115123125783e-17

Calculate:

$$s = \sum_{i=1}^{1000} \frac{i}{1000}$$

```

a = 1/1000
b = 0.0
s = 0.0
for i in range(1000):
    b += a
    s += b
print(s)

```

500.500000000000034

#always use the direct formula:

```

a = 1/1000
b = 0.0
s = 0.0
for i in range(1000):
    b = (i+1) * a
    s += b
print(s)

```

500.5

Most famous example: The Minecraft boat-drop mystery ([external video](#))

Canonical example exponential

The exponential function

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Get the next element from the previous one:

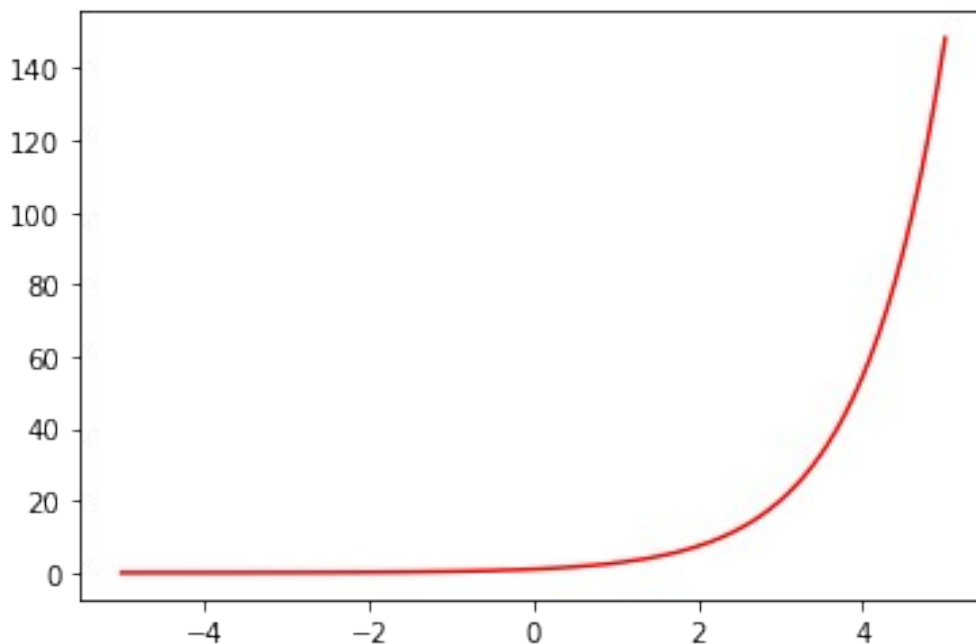
$$\frac{x^n}{n!} = \frac{x}{n} \cdot \frac{x^{n-1}}{(n-1)!}$$

If we create a loop and have a temporary variable `vn1`, then it is enough to multiply it by x/n to get the new one:

$$vn1 := vn1 * x / n$$

Implement it!

```
x = np.linspace(-5,5,100)
plt.plot(x,np.exp(x),"r-")
[<matplotlib.lines.Line2D at 0x7f3c339138b0>]
```



```
def myexp(x):
    #initial value
    vn1 = 1.0
    e = 1.0
    prev_e = -1.0 # previous value used for stop contition
    n = 1
    while np.abs(prev_e - e) > 0:
        prev_e = e #save the previous value
        vn1 *= x / n # the new value of the sum
        e += vn1 # add it!
        n += 1 # increase index
    return e
```

```
print(np.exp(0.1), myexp(0.1))
1.1051709180756477 1.1051709180756473
print(np.exp(20), myexp(20))
485165195.4097903 485165195.40979046
print(np.exp(-20), myexp(-20), 1.0/myexp(20))
2.061153622438558e-09 6.147561828914626e-09 2.061153622438557e-09
```

Numpy

`numpy` is the most general vector and linear algebra tool in python. It includes the most common operations of linear algebra, complex numbers, mathematical functions and random numbers. I reiterate that do not mix with the `math` and `random` package. They have different number styles, and different precisions. Python is a mess by itself there is no need to make your life harder.

```
a = np.arange(10)
b = range(10)
print(a, b, list(b))
print(type(a), type(b))
#please note the subtle difference of missing commas. Do not rely on it!

[0 1 2 3 4 5 6 7 8 9] range(0, 10) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
<class 'numpy.ndarray'> <class 'range'>

#numpy array and list can be made from each other
b = [1, 2, 3]
print(type(b), b, b[0])
a = np.array(b)
print(type(a), a, a[0])
c = list(a)
print(type(c), c, c[0])

<class 'list'> [1, 2, 3] 1
<class 'numpy.ndarray'> [1 2 3] 1
<class 'list'> [1, 2, 3] 1

#numpy cannot mix types, if the list does it will take the most inclusive one, here the string
np.array([1, "a", 1.5])
#here float
np.array([1, 1.5, 7])
array([1. , 1.5, 7. ])
```

#very efficient in storage. I have 8 byte integers by default:

```
a = np.arange(10000)
b = np.arange(100000)
print(sys.getsizeof(a),sys.getsizeof(b))
```

```
80096 800096
```

#list first stores a list of pointers to the elements which are in turn stored individually in the memory.

```
la = [i for i in range(10000)]
print(sys.getsizeof(la))
print(sys.getsizeof(la[0]))
```

```
87616
```

```
24
```

```
la = ["A LONG STRING THAT DEFINITELY"+str(i)+" DOES NOT FIT" for i in
range(10000)]
print(sys.getsizeof(la))
print(sys.getsizeof(la[0]))
```

```
87616
```

```
92
```

#the extra space stores the shape of the numpy array. reshaping is cheap:

```
a = np.array([1,2,3,4,5,6])
b = np.array([[1,2,3],[4,5,6]])
print(np.ndim(a))
print(np.ndim(b),np.ndim(b[0]))
print(a.reshape(2,3),a.reshape(2,3).ndim)
```

```
1
```

```
2 1
```

```
[[1 2 3]
```

```
 [4 5 6]] 2
```

#obviously:

```
a = np.array([1,2,3,4,5,6])
c = a.reshape(2,4)
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
<ipython-input-10-1093854b4323> in <module>
```

```
1 #obviously:
```

```
2 a = np.array([1,2,3,4,5,6])
```

```
----> 3 c = a.reshape(2,4)
```

```
ValueError: cannot reshape array of size 6 into shape (2,4)
```

#zeros is the default way to create a numpy array. Always define the type!!!!!!! Though default is float.

```
a = np.zeros((3,3))
b = np.zeros((3,3),dtype=int)
print(a)
print(b)
c = np.ones(5,dtype=np.uint8) #8 bit unsigned integer
print(c)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[0 0 0]
 [0 0 0]
 [0 0 0]]
[1 1 1 1 1]
```

#indexing, and slicing

```
a = np.zeros((3,3),dtype=int)
print("a=",a)
a[0][1] = 1
a[1, 2] = 3.5 # just to show that it will be converted back to integer
print("\na=",a)
a[:,0:3:2] = 2 # slicing. The single : means all
rows/columns/whatever comes after it in the third dimension
print("\na=",a)
```

```
a= [[0 0 0]
     [0 0 0]
     [0 0 0]]
```

```
a= [[0 1 0]
     [0 0 3]
     [0 0 0]]
```

```
a= [[2 1 2]
     [2 0 2]
     [2 0 2]]
```

```
c[0] += 255
print(c)
#it is zero as a 8 bit unsigned goes from 0 to 255
```

```
[0 1 1 1 1]
```

operations on the whole array or on a part of it

```
a = np.zeros((3,3),dtype=float)
a[1] += 1
print(a)
a[:,2] += 2
print(a)
```

```

[[0. 0. 0.]
 [1. 1. 1.]
 [0. 0. 0.]]
[[0. 0. 2.]
 [1. 1. 3.]
 [0. 0. 2.]]

a = a*(-1)
print(a)

[[-0. -0. -2.]
 [-1. -1. -3.]
 [-0. -0. -2.]]

#but
a[:2,::2] = a[:2,::2] + 10
print(a)

[[10. -0.  8.]
 [ 9. -1.  7.]
 [-0. -0. -2.]]

a[:2,::2] += np.ones((2,2))*10
print(a)

[[20. -0. 18.]
 [19. -1. 17.]
 [-0. -0. -2.]]

# columns and rows
a = np.arange(9).reshape(3,3)
print(a)
print("")
print(a[1])
print(a[:,1])

[[0 1 2]
 [3 4 5]
 [6 7 8]]

[3 4 5]
[1 4 7]

```

Random numbers

All random numbers created by python are pseudo random numbers which means they are created by an algorithm, which was proven to fullfill all requirements of randomness. The new numbers are always created from the last generated one. So there must be a first one which is called seed. If you omit it the computer will use the current time. Advantages of using your own seed:

- You can debug your code

- Results are reproducible
- If by chance you run a parallel code. All instances generate the same random sequence.

```

np.random.seed(12345)
a = np.random.random(10)
print(a)

[0.92961609 0.31637555 0.18391881 0.20456028 0.56772503 0.5955447
 0.96451452 0.6531771 0.74890664 0.65356987]

#events with given probability. Here 30%
N = 10
p = 0.3
for i in range(N):
    if np.random.random() < p:
        print("It happened in step %d." % (i))

It happened in step 2.
It happened in step 3.
It happened in step 4.

# do it in one step
p = 0.3
r = np.random.random(10) < p
print(r)

[False False False False False False False False True]

#the True/False array can be used as an index to restrict the other
array to the True part
np.arange(10)[r]

array([9])

np.where(r)

(array([9]),)

# Other useful random functions:
print(np.random.choice(5, 10))
print(np.random.choice(5,3,replace=False))
print(np.random.choice(5, 10, p=[0.1, 0, 0.3, 0.6, 0]))
c = ["Budapest", "Pécs", "Debrecen", "Miskolc"]
print(np.random.choice(c,1))
print(np.random.choice(c,2))
sr = np.arange(5)
np.random.shuffle(sr)
print(sr)

[3 1 3 1 3 4 0 0 3 2]
[0 1 4]
[3 2 2 3 3 2 0 2 3 2]

```

```

['Budapest']
['Miskolc' 'Budapest']
[0 1 3 2 4]

# shape of the array
np.random.seed(12345)
a = np.random.random(size=(2,5))
print(a)

[[0.92961609 0.31637555 0.18391881 0.20456028 0.56772503]
 [0.5955447  0.96451452 0.6531771  0.74890664 0.65356987]]

```

copying

```

a = np.arange(5)
b = a
b[3] = 9
print(a,b) #they are the same

[0 1 2 9 4] [0 1 2 9 4]

a = np.arange(5)
b = a.copy()
b[3] = 9
print(a,b) #they are not the same

[0 1 2 3 4] [0 1 2 9 4]

a = np.arange(6).reshape(3,2)
b = a[0:2]
b[1,1] = 9
print(a,"\n",b) #they are not the same

[[0 1]
 [2 9]
 [4 5]]
[[0 1]
 [2 9]]

```

Functions

```

def f1(x):
    x = x + 1
    return x

x = 2
print(f1(x),x)

3 2

```

```
def f2(l): #changes also the original
    l[0] += 1
    return(l)
```

```
l = [1,2,3]
print(f2(l),l)
```

```
[2, 2, 3] [2, 2, 3]
```

```
f3 = lambda x: x%7
l = [1,3,7,9]
print(sorted(l, key = f3))
```

```
[7, 1, 9, 3]
```

```
def addfirst(x, getfirst = lambda x:x):
    return getfirst(x) + 1
```

```
addfirst(5)
```

```
6
```

```
addfirst([5,4,3])
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
<ipython-input-111-67c125152da8> in <module>
----> 1 addfirst([5,4,3])
```

```
<ipython-input-108-6757c834e50b> in addfirst(x, getfirst)
      1 def addfirst(x, getfirst = lambda x:x):
----> 2     return getfirst(x) + 1
```

```
TypeError: can only concatenate list (not "int") to list
```

```
addfirst([5,4,3],getfirst = lambda x:x[0])
```

```
6
```

Random number example

Typical problem: We have an array with many values and we have to change them randomly with a given probability p

First for loop

```
N = 1000000
p = 0.00001
a = np.zeros(N,dtype=float)
```



```

for i in range(N):
    if np.random.random() < p:
        a[i] = np.random.normal(0,0.1)

%%timeit
a = np.zeros(N,dtype=float)
for i in range(N):
    if np.random.random() < p:
        a[i] = np.random.normal(0,0.1)

271 ms ± 3.99 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

a = np.zeros(N,dtype=float)
r = np.random.random(N) < p
a[r] = np.random.normal(0,0.1,size=r.sum())

%%timeit
a = np.zeros(N,dtype=float)
r = np.random.random(N) < p
a[r] = np.random.normal(0,0.1,size=r.sum())

5.92 ms ± 87.1 µs per loop (mean ± std. dev. of 7 runs, 100 loops
each)

n = np.random.binomial(N,p)
print(n)

6

```

Idea: We "know" how many number will be changed, or at least the distribution of it: binomial. So we generate a number with binomial distribution and only that many random numbers and positions. The problem with python is that the positions is tricky to get, as we have a terrible implementation of `np.random.choice` with the `replace=False` option.

```

%%timeit
a = np.zeros(N,dtype=float)
n = np.random.binomial(N,p)
a[np.random.choice(N,n,replace=False)] =
np.random.normal(0,0.1,size=n)

13 ms ± 176 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

%%timeit
a = np.zeros(N,dtype=float)
n = np.random.binomial(N,p)
a[np.random.choice(N,n)] = np.random.normal(0,0.1,size=n)

272 µs ± 6.43 µs per loop (mean ± std. dev. of 7 runs, 1000 loops
each)

```